

# SSI Implementation Status and Validation Findings

## Pass 2: branch reconstruction, persistence, and frequency-tolerance validation

### Summary for supervision discussion

This note summarises the SSI implementation built so far and the empirical findings from validating it against synthetic, known-truth data. It follows on from the earlier SSI method summary; here the focus is on what has actually been built, tested, and learned.

## 1 Implementation status

| File                                    | Purpose                                             | Status                                                 |
|-----------------------------------------|-----------------------------------------------------|--------------------------------------------------------|
| <code>buildHankelMatrix.m</code>        | Past/future block Hankel construction               | Validated (exact, hand-computed case)                  |
| <code>hankelProjection.m</code>         | QR-based projection<br>$P = Y_f / Y_p^{\text{ref}}$ | Validated (exact vs. direct formula)                   |
| <code>extractStateSpace.m</code>        | SVD extraction of $(A, C)$ at order $n$             | Validated (known system + twin test)                   |
| <code>modalParameters.m</code>          | Eigenvalue $\rightarrow (\omega, \zeta, \Phi)$      | Validated; branch-cut bug found and fixed              |
| <code>computeMAC.m</code>               | Modal Assurance Criterion                           | Validated; third-party squared-numerator bug ruled out |
| <code>sweepModelOrders.m</code>         | Per-order extraction, conjugate dedup               | Validated (multi-mode system, hard invariant check)    |
| <code>classifyPoleStability.m</code>    | Order-to-order matching, class 0–3                  | Validated (full hierarchy, one-to-one assignment)      |
| <code>buildModalBranches.m</code>       | Reconstructs order-tracked pole branches            | Validated (known structure + integration test)         |
| <code>computeBranchPersistence.m</code> | Sliding-window persistence criterion                | Validated; parameters shown to be context-dependent    |

**Architecture.** The pipeline is deliberately layered so that no single function is responsible for deciding whether a pole is physically real:

Hankel  $\rightarrow$  projection  $\rightarrow$  state space at order  $n \rightarrow$  modal parameters  
 $\rightarrow$  pairwise classification  $\rightarrow$  branches  $\rightarrow$  persistence.

Persistence answers only “is this a numerically consistent candidate”; physical interpretation (floe mode vs. radiation-memory pole vs. forcing-colour artifact) is deliberately left to a later stage, not built into any function here.

## 2 A real bug caught by a structural invariant

**Branch-cut artifact in `modalParameters.m`.** A real, *negative* discrete eigenvalue produces a spurious  $+i\pi/\Delta t$  term after `log()` (MATLAB’s complex branch cut), which looks exactly like a genuine oscillatory pole with no conjugate partner. This was not caught by any plausibility check; it was caught because `sweepModelOrders.m`’s own tester asserted a hard mathematical bound (a real system of order  $n$  can have at most  $\lfloor n/2 \rfloor$  conjugate pairs), and the phantom pole violated it at every odd order. Fixed by testing `imag(lambda)` (the discrete eigenvalue, before the branch cut) rather than `imag(lambda_c)` (after it). This is a concrete example of why hard invariant checks, not just plausibility checks, were built into this project’s testers throughout.

## 3 Pass 1 integration test: noise-free vs. noisy

A synthetic 3-mode system ( $\omega = 1.2, 2.7, 4.1$  rad/s, single output channel, true state order 6) was swept from model order 1 to 14.

**Noise-free case: exact recovery.** All three true modes were recovered to the displayed precision (matching  $\omega_{\text{true}}$  exactly) from order 6 onward, remaining exactly stable at every higher order tested, with an unbroken run of class-3 links from order 6 through 14. This confirms the full chain (Hankel through classification) is internally consistent end to end.

**Noisy case (1% of signal std): degradation, not failure.** The same three modes now drift across orders and the class-3 links become intermittent rather than unbroken. Critically, all three modes still showed genuine class-3 links repeatedly, just not consecutively. This directly motivated `computeBranchPersistence.m`’s sliding-window design (“ $k$  of  $W$ ” rather than strict consecutiveness), since a strict consecutive-run requirement was shown to under-report modes that are visibly, repeatedly stable but not in an unbroken

sequence.

## 4 Branch fragmentation and its cause

Applying the persistence criterion directly to the noisy 3-mode data initially showed *no* persistent branches at all, for a subtler reason than weak stability: `buildModalBranches.m` deliberately does not bridge gaps (an unmatched pole ends a branch; a later reappearance starts a new one). Under noise, some genuine modes were fragmenting into several short branches, each individually too short to ever satisfy the window-size requirement, regardless of how permissive the persistence threshold was set.

**Diagnosis via a controlled `freqTol` sweep.** Rather than adjusting `computeBranchPersistence.m`, `buildModalBranches.m`, or `classifyPoleStability.m` to mask the symptom, the raw per-order frequency estimates were inspected directly. For two of the three true modes (1.2 and 2.7 rad/s), a compatible pole was present at every order, drifting by more than the 1% frequency gate allowed between some consecutive orders. Widening the gate to 5% reconnected these into single, unbroken, fully class-3 branches spanning most of the sweep, with no change to any other function. For the third mode (4.1 rad/s), no compatible pole was found near it at *any* tolerance tested, at any order: this is not a gating problem at all, but an apparent estimation failure under this noise realisation.

**Open: the missing 4.1 rad/s mode.** Not yet explained. Candidate explanations include the smaller modal weighting given to this mode in the synthetic output matrix, this specific noise realisation, finite record length, or numerical conditioning. Not established as a fundamental observability limit; requires testing across multiple noise seeds before drawing a conclusion.

## 5 The MAC-triviality finding and its consequence for sensor count

**MAC provides zero discrimination with a single sensor channel.** For any two nonzero scalars  $a, b$  (an  $l = 1$  “mode shape”),  $\text{MAC}(a, b) = |a^*b|^2 / (|a|^2|b|^2) = 1$  identically, regardless of how different  $a$  and  $b$  are. Confirmed numerically for several arbitrary complex scalar pairs. This means every single-channel test run so far has effectively used frequency and damping matching only; the MAC gate in `classifyPoleStability.m` could never fail in any of those tests, by mathematical necessity rather than coincidence.

This directly motivated a controlled experiment on whether the `freqTol = 5%` tolerance validated above is safe in general, or only safe because the 3-mode test case happened to have well-separated frequencies.

**Experiment: two close true modes (2.70, 2.80 rad/s, 3.7% separation).** With a single sensor channel, `freqTol = 5%` produced branches that visibly oscillated between the neighbourhoods of both true frequencies rather than settling on either one: a genuine wrongful merge, not a borderline case. With a second sensor channel, chosen with different modal weighting between the two modes (making MAC meaningful), the identical tolerance cleanly separated the two modes into distinct branches, each locking exactly onto its own true frequency for eleven consecutive orders with no confusion.

**Consequence: `freqTol` is not a single universal constant.** A tolerance that correctly rescues genuine noisy modes in one configuration can wrongly merge genuinely distinct modes in another, depending on how many sensor channels are available and how distinct their modal sensitivity is. The appropriate value requires validation against the actual sensor configuration eventually used, not a value fixed in advance.

This result is a direct, independent piece of software-level evidence for the multiple-sensor decision already made on other grounds (identifiability of  $D, R, E^v$ ): with only one sensor, the SSI pipeline’s own mode-separation mechanism has no protection against merging distinct, closely-spaced modes once the frequency tolerance is widened enough to tolerate realistic noise.

## 6 Recommended next steps

1. `syntheticSensorData.m`: the bridge between the validated Forward Model (Phase 1, including the JONSWAP extension) and this SSI pipeline. Given a precomputed deflection dataset and a list of sensor locations  $(r_i, \theta_i)$ , produces the genuine multi-channel time series matrix every function above already expects.
2. Repeat the close-mode experiment (Section 5) using genuine multi-point Forward Model output rather than hand-built oscillators, at the sensor count and rough placement already agreed (3–4 sensors).
3. Investigate the missing 4.1 rad/s mode across multiple noise seeds before concluding anything about its observability.
4. Defer any change to `buildModalBranches.m`’s gap-bridging behaviour: the fragmentation observed here was

fully explained by the frequency-tolerance mechanism, not a deficiency in branch reconstruction itself.